REGULARIZATION

Lasso, Ridge Regression, and Elastic Nets

Marco R. Steenbergen

steenbergen@ipz.uzh.ch

University of Zurich

July 20, 2026

R Packages for this Lecture

- `glmnet` for regularized linear and logistic regression.

What Is Regularization?

The Basic Idea

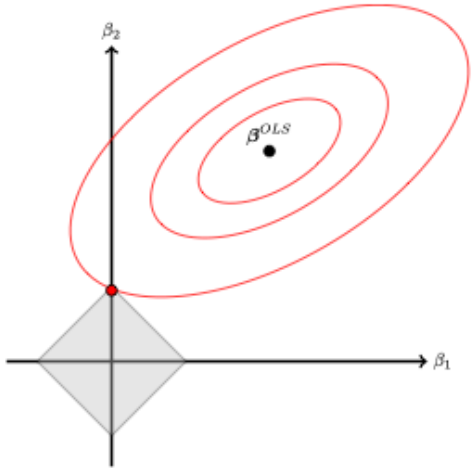
- **Regularization** adds a penalty for over-fitting to the loss function.
- This penalty serves as a constraint on the optimization problem.
- We can also think of this as a shrinkage estimator, with small coefficients being shrunk more than larger coefficients.
- The most important regularization procedures are
    1. The lasso.
    2. Tikhonov regularization/ridge regression.
    3. Elastic nets.

The Lasso

- **Lasso** = least absolute shrinkage and selection operator (Tibshirani 1996).
- We impose $\sum_{j=1}^{P} |\beta_j| \leq t$.
- Using Lagrange multipliers, the lasso loss is $L_{\text{lasso}} = L_2 + \lambda \left(\sum_{j=1}^{P} |\beta_j| - t \right)$.
- Retaining only the terms in ω, the optimization problem is

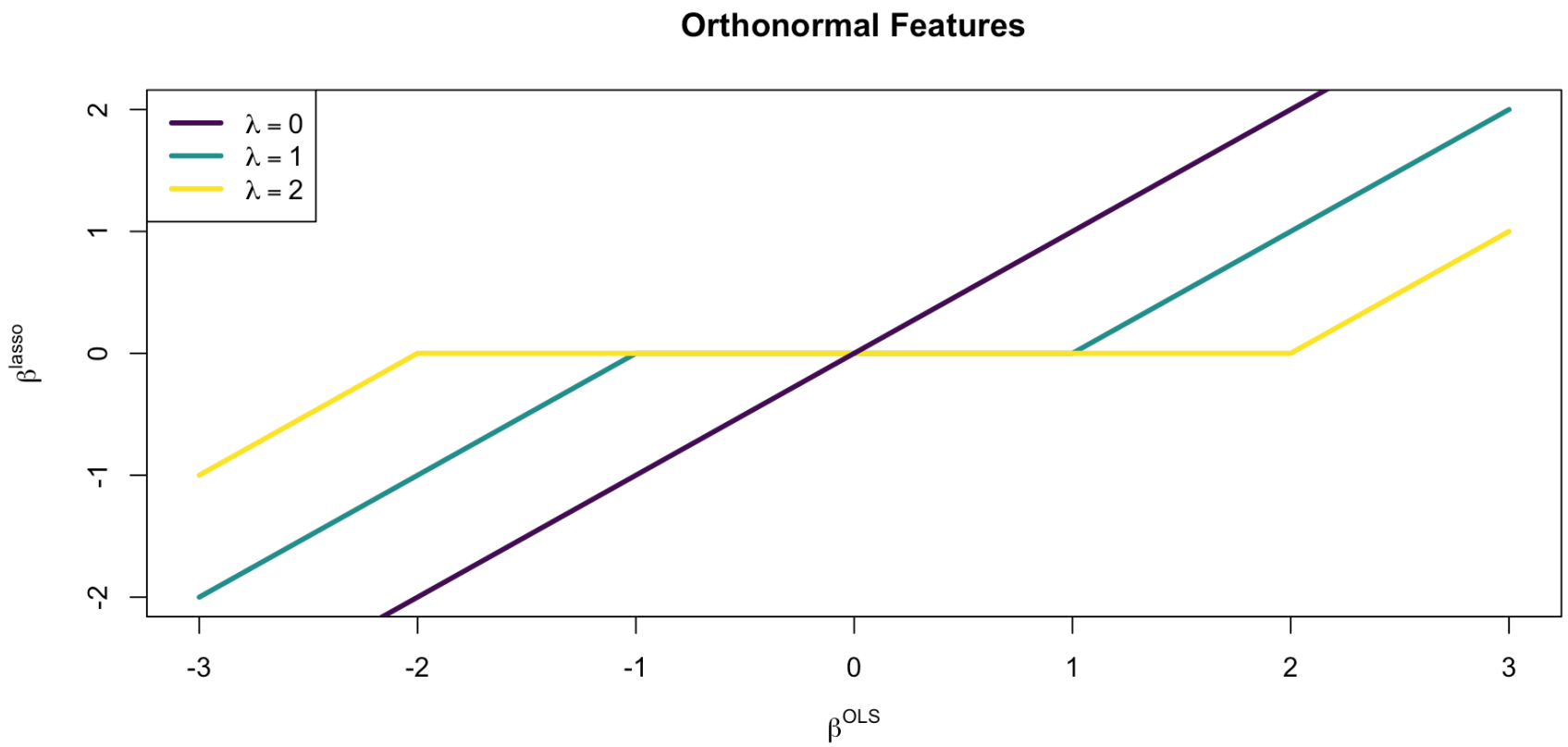
$$\min_{\alpha, \boldsymbol{\beta}} \sum_{i=1}^{n_1} \left(\alpha + \boldsymbol{x}_i^\top \boldsymbol{\beta} - y_i \right)^2 + \lambda |\boldsymbol{\beta}|$$

- λ is a hyper-parameter.



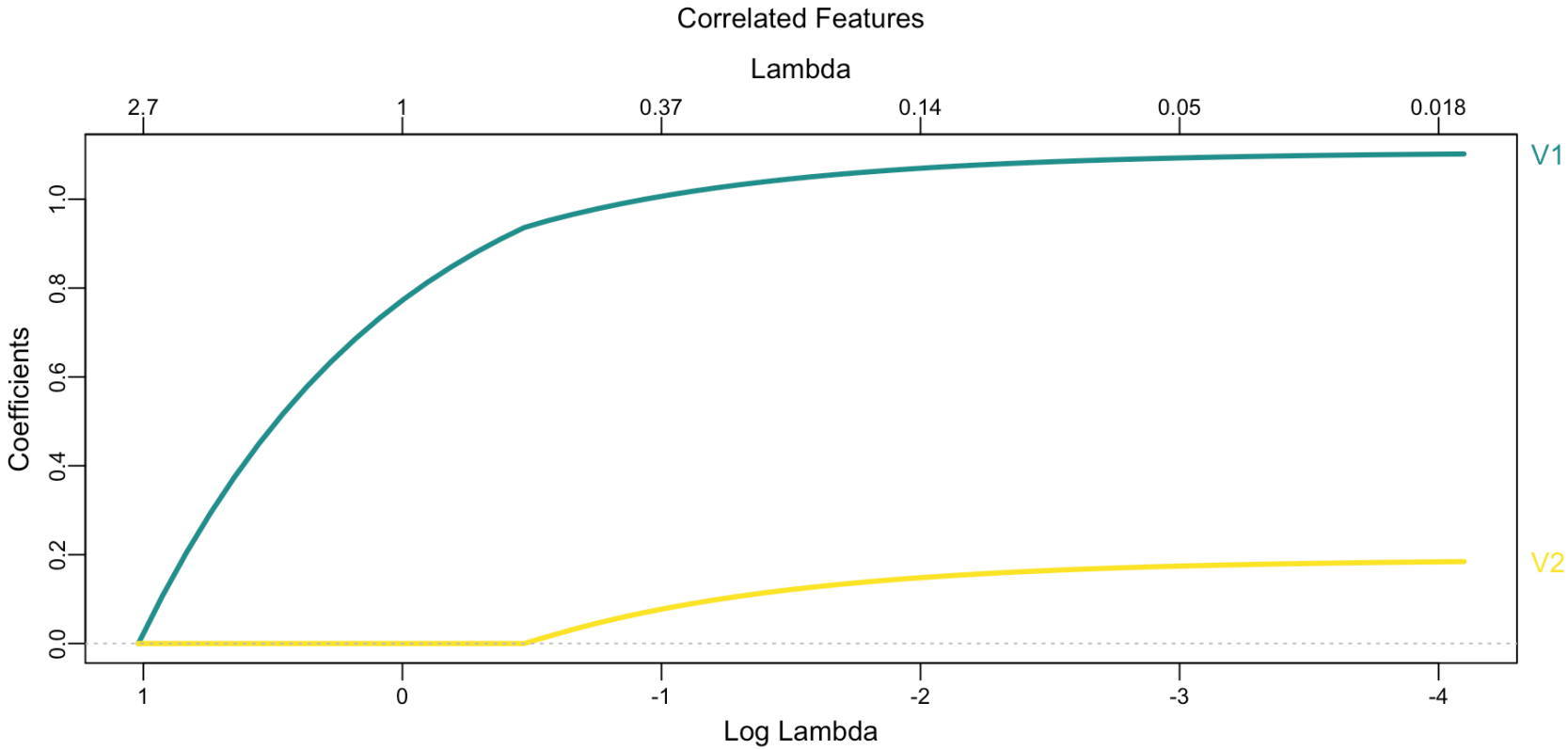
What the Lasso Does

1st Example | 2nd Example



What the Lasso Does

1st Example | 2nd Example



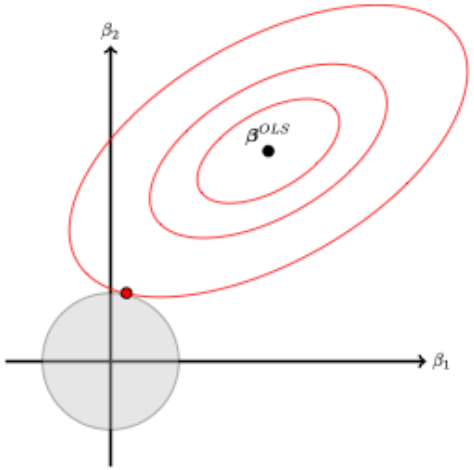
Variations on a Theme

- We can think of the lasso as a special case of

$$\min_{\alpha,\boldsymbol{\beta}} \sum_{i=1}^{n_1} \left(\alpha + \boldsymbol{x}_i^\top \boldsymbol{\beta} - y_i\right)^2 + \lambda |\boldsymbol{\beta}|^q$$

- For $q = 1$, this becomes the lasso.
- For $q = 2$, this becomes **Tikhonov regularization** (a.k.a. ridge regression).

Tikhonov Regularization



- We minimize the L_2-norm subject to $\sum_{j=1}^{P} \beta_j^2 \leq t$ (Tikhonov 1963).
- Hence,

$$\min_{\alpha,\beta} \sum_{i=1}^{n_1} \left(\alpha + \boldsymbol{x}_i^\top \boldsymbol{\beta} - y_i\right)^2 + \lambda \sum_{j=1}^{P} \beta_j^2$$

Elastic Nets

- Elastic nets are a mixture of the lasso and Tikhonov regularization (Zou and Hastie 2005).
- For $\gamma \in [0,1]$, define the penalty

$$P(\gamma) = \sum_{j=1}^{P} \left[\frac{1}{2}(1-\gamma)\omega_j^2 + \gamma|\omega_j| \right]$$

- Then

$$\min_{\boldsymbol{\omega}} \sum_{i=1}^{n_1} \left(\beta + \boldsymbol{x}_i^\top \boldsymbol{\omega} - y_i \right)^2 + \lambda P(\gamma)$$

Elastic Nets Cont'd

- Note this reduces to
  - → the lasso for $\gamma = 1$.
  - → Tikhonov regularization for $\gamma = 0$.
- Here γ is another hyper-parameters.

How to Select and Validate Hyper-Parameters

A Wealth of Possibilities

- In rare cases—e.g., principal component analysis—hyper-parameters take on a finite number of discrete values $\Rightarrow$ grid search: try each value.
- This is not our case: γ and λ are both continuous and there are endless combinations.
- The simplest possibility is to do a random search:[1]

  → Max entropy when there is 1 hyperThe-parameter.

  → Latin hyper-cubes with max entropy when there are multiple hyper-parameters.

1. SMBO will be introduced later in the course.

Space-Filling and Latin Hyper-cubes

- **Goal:** For a K-dimensional parameter space, we must ensure uniform coverage across all dimensions.
  - → Standard random sampling leaves empty voids and redundant clusters.
  - → Space-filling designs (like Latin Hyper-cubes) force points to spread evenly (Husslage et al. 2011; McKay et al. 1979).
- **Maximum Entropy Designs** (e.g., Kriging / Gaussian Process approach):
  - → Model hyperparameter locations as a spatial process where points are correlated.
  - → The `variogram_range` parameter controls the spatial decay / effective repulsion scale.
  - → **Higher `variogram_range` values** expand the repulsion field, reducing the likelihood of large empty voids in the space.

Evaluating the Grid

- Now that we have set the grid, how do we evaluate different choices of hyper-parameter values?
- The basic idea is this:
  1. Set (γ, λ).
  2. Train the model with those values.
  3. Evaluate performance without touching the test data—they come at the very end.
- The problem: resubstitution error (Efron 1983).
- We need to set aside data within the training set to **validate** our choice of hyper-parameters.

Building Validation Sets

1. The validation split.
2. v-Fold cross-validation.
3. Bootstrapping.

The Validation Split

- Take the n_1 training instances.
- Set a probability of q for training use and $1 - q$ for validation use:

  →

  $q \cdot n_1$ training instances proper—used to estimate $\boldsymbol{\theta}|\boldsymbol{\tau}$, where $\boldsymbol{\tau}$ is a set of hyper-parameter values from the grid.

  →

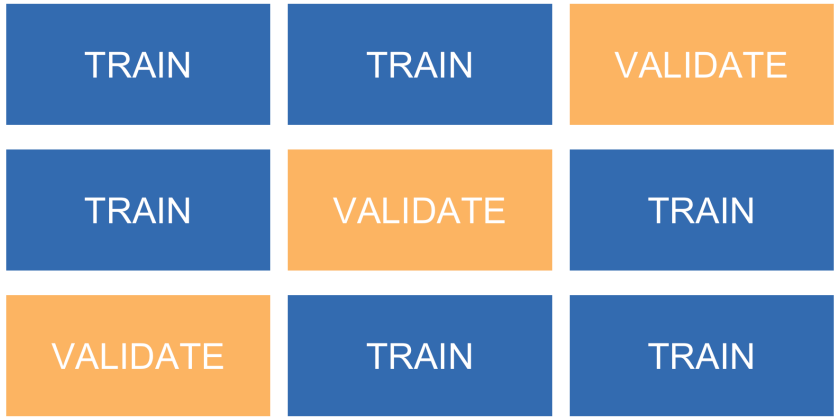
  $(1 - q) \cdot n_1$ validation instances—used to assess predictive performance given $\boldsymbol{\tau}$.

- We iterate over the grid and pick as $\boldsymbol{\tau}$ that set of values that maximize performance in the **validation set**.

The Validation Split Cont'd

- Advantage: Only have to train once per combination of hyper-parameters.
- Disadvantages:
  1. We lose training data: $q \cdot n_1 < n_1$.
  2. We may end up with an atypical validation set.
  3. We obtain only one performance value per combination.
- In large samples, the first two problems lose relevance and the computational advantage often outweighs the third disadvantage.
- In medium-sized data, the disadvantages loom larger (Molinaro et al. 2005).
- But can we do better?

v-Fold Cross-Validation



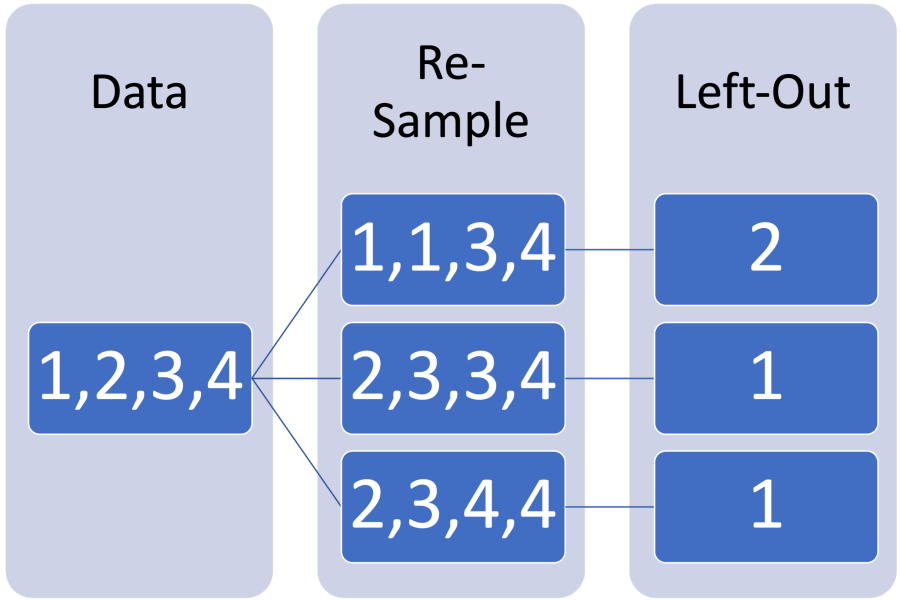
v-Fold Cross-Validation steps:

- Randomly assign instances to k folds (typically, $v = 10$).
- Each fold is held out once for validation purposes $\Rightarrow v$ performance estimates per τ.
- Each fold is used $v - 1$ times for training purposes $\Rightarrow$ no loss of training data.

Variations on a Theme

- The higher we set k, the more data will be used for training at any given time.
- An extreme case is **leave out one CV** (LOOCV), where $k = n_1 - 1$.
- In each run, bias is reduced but variance is increased.
- Another way to reduce bias is to **repeatedly** assign instance to the folds, each time using a different seed.
- **Monte Carlo CV** selects the folds randomly each time, which may cause folds to contain overlapping instances (Xu and Liang 2001).

Bootstrapping



- The **bootstrap** consists of repeatedly sampling n_1 instances with replacement (Efron 1979).
- Sampled instances are used for training.
- Non-sampled instances are used for validation.

References

Efron, Bradley. 1979. “Bootstrap Methods: Another Look at the Jackknife.” *The Annals of Statistics* 7 (1): 1–26. http://www.jstor.org/stable/2958830.

Efron, Bradley. 1983. “Estimating the Error Rate of a Prediction Rule: Improvement on Cross-Validation.” *Journal of the American Statistical Association* 78 (382): 316–31. https://doi.org/10.1080/01621459.1983.10477973.

Husslage, Bart G. M., Gijs Rennen, Edwin R. Van Dam, and Dick Den Hertog. 2011. “Space-Filling Latin Hypercube Designs for Computer Experiments.” *Optimization and Engineering* 12 (4): 611–30. https://doi.org/10.1007/s11081-010-9129-8.

McKay, M. D., R. J. Beckman, and W. J. Conover. 1979. “Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code.” *Technometrics* 21 (2): 239–45. https://doi.org/10.1080/00401706.1979.10489755.

Molinaro, Annette M., Richard Simon, and Ruth M. Pfeiffer. 2005. “Prediction Error Estimation: A Comparison of Resampling Methods.” *Bioinformatics* 21 (15): 3301–7. https://doi.org/10.1093/bioinformatics/bti499.

Tibshirani, Robert. 1996. “Regression Shrinkage and Selection Via the Lasso.” *Journal of the Royal Statistical Society Series B: Statistical Methodology* 58 (1): 267–88. https://doi.org/10.1111/j.2517-6161.1996.tb02080.x.

Tikhonov, Andrey. 1963. “Solution of Incorrectly Formulated Problems and the Regularization Method (О Решении Некорректно Поставленных Задач и Методе Регуляризации).” *Soviet Mathematics Doklady* 5: 1035–38.

Xu, Qing-Song, and Yi-Zeng Liang. 2001. “Monte Carlo Cross Validation.” *Chemometrics and Intelligent*